



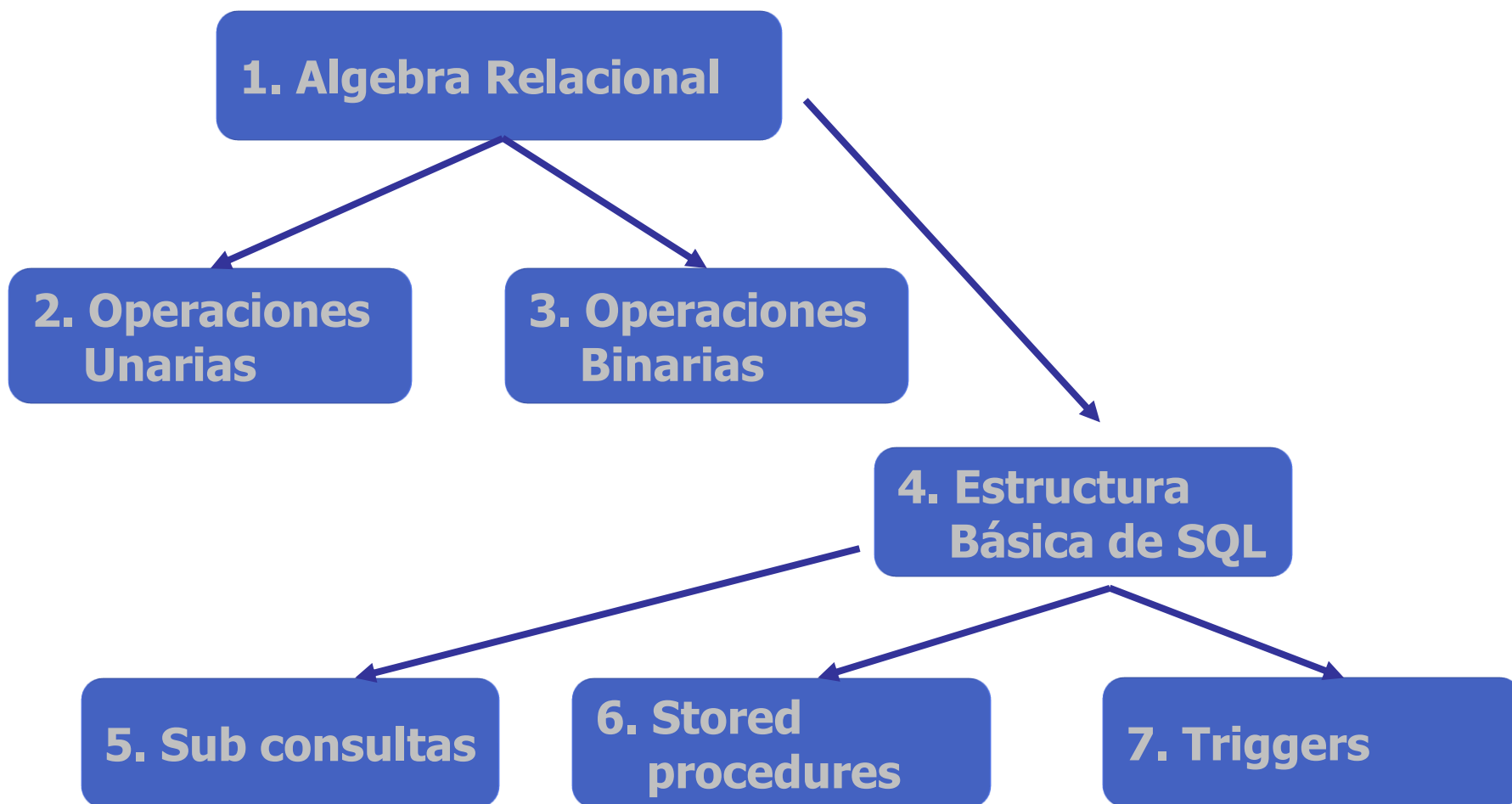
Facultad de Ciencias Económicas
Universidad de Buenos Aires



Diseño y gestión de base de Datos

UT 4: Lenguajes de consulta y actualización de base de datos

Esquema de la clase



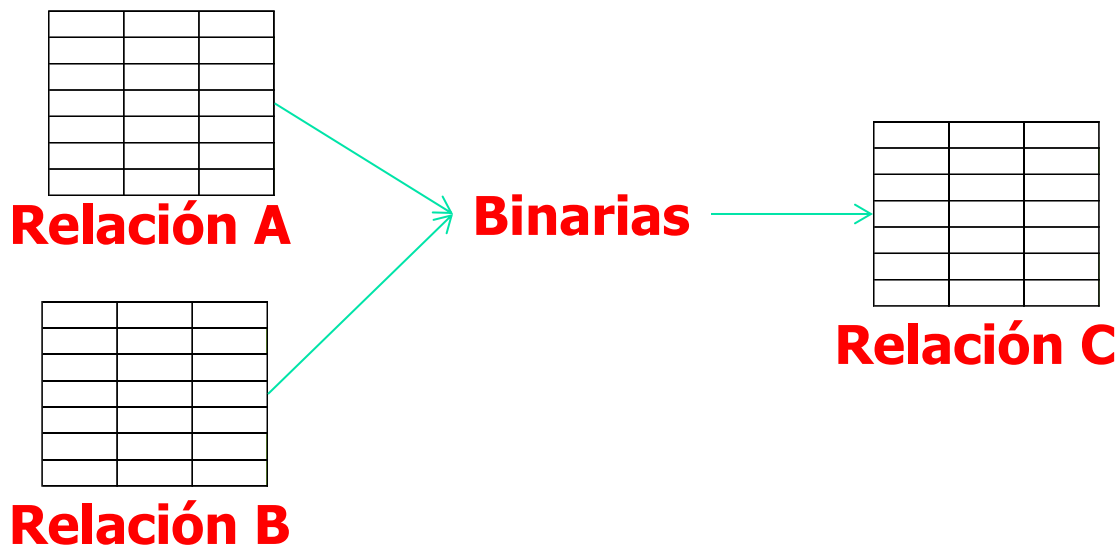
Alegra Relacional

¿Qué es el Álgebra Relacional?

- Es un lenguaje de consulta procedimental.
- Conjunto de operaciones que describen paso a paso como computar una respuesta sobre las relaciones definidas en el modelo relacional.
- Relacionadas con SQL-DML (manipulación de datos)
- Se usan como una representación intermedia de una consulta a una base de datos
- Permite obtener una versión más optimizada y eficiente



Operaciones del Álgebra Relacional



Operaciones del Álgebra Relacional

- Operaciones Fundamentales del Álgebra Relacional

- Selección (σ)
- Proyección (Π)
- Renombramiento (ρ)

Unarias

- Unión ($\cup$)
- Diferencia ($-$)
- Producto cartesiano ($\times$)

Binarias

Relación/Entidad (R)
Condición ($P_1, P_2, \dots, P_n$)
Atributos ($A_1, A_2, \dots, A_n$)

- Otras Operaciones

- Intersección ($\cap$)
- Reunión Natural ($\bowtie$)
- Reunión Externa ($\lrcorner \bowtie \lrcorner$)



Álgebra Relacional – Selección

Selección (σ)

Permite seleccionar un subconjunto de tuplas de una relación (R), todas aquellas que cumplan la(s) condición(es) P:

$$\sigma_p(\mathbf{R})$$

Por Ej:

$\sigma_{\text{nombre_escuela}} = \text{"Normal 5"}(\text{Escuela})$

[illegible]

Proyección (Π)

Permite extraer columnas (atributos) de una relación, dando como resultado un subconjunto vertical de atributos de la relación

$$\prod_{A_1, A_2, \dots, A_n} (R)$$

Por Ej:

Π_{codigo_escuela,nombre_escuela} (Escuela)

[illegible]

Álgebra Relacional – Renombramiento

Renombramiento (ρ)

Permite poner nombre a las expresiones del Álgebra relacional (renombrar las tablas en las operaciones)

$\rho_A(\mathbf{R})$

Cliente

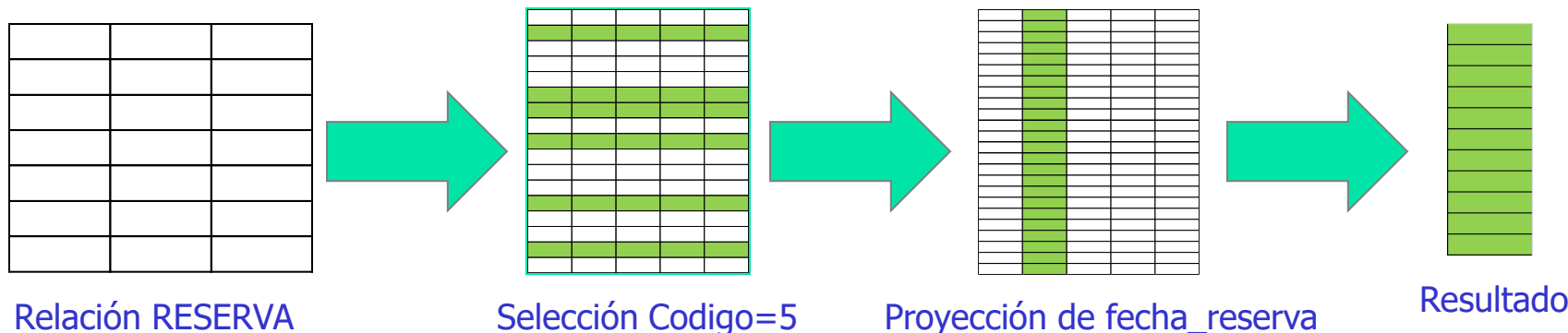
CLI

Por Ej:

$\rho_{res}(\mathbf{Reserva})$

Ejemplo

Obtener las fechas de las reservas de la escuela con código 5



Relación RESERVA

Selección Codigo=5

Proyección de fecha_reserva

Resultado

$\Pi_{\text{fecha_reserva}}(\sigma_{\text{Codigo_Escuela}=5}(\text{Reserva}))$

SELECT Fecha_Reserva FROM Reserva WHERE código_escuela = 5

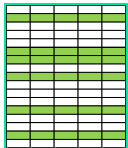
Álgebra Relacional -> SQL

3 SELECT Fecha_Reserva

1 FROM Reserva

2 WHERE Codigo_Escuela = 5

1 Identificación de las Relaciones involucradas

2 Selección (σ)  $\sigma_{\text{Codigo_Escuela}=5}(\text{Reserva})$

3 Proyección (Π)  $\Pi_{\text{fecha_reserva}}(\text{Reserva})$

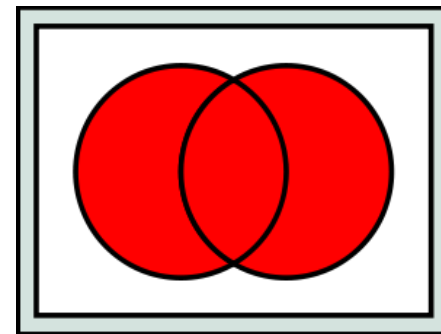
$\Pi_{\text{fecha_reserva}}(\sigma_{\text{Codigo_Escuela}=5}(\text{Reserva}))$

Álgebra Relacional – Unión

Unión (U)

Retorna el conjunto de tuplas que están en R, o en S, o en ambas. R y S deben ser relaciones compatibles:

R U S



Por Ej:

Reserva U Reserva_Historica

Álgebra Relacional -> SQL

Unión (U)

```
SELECT Cod_Reserva, Fecha  
FROM Reserva  
WHERE Fecha = '30/04/2009'
```

UNION

```
SELECT Cod_Reserva, Fecha_Hist  
FROM Reserva_Historica  
WHERE Fecha_Hist = '01/04/2005'
```

Relaciones compatibles:

- **Cantidad de Atributos**
- **Dominios Compatibles**

$\Pi_{\text{Cod_Reserva, Fecha}} (\sigma_{\text{Fecha} = '30/04/2009'} (\text{Reserva}))$
U

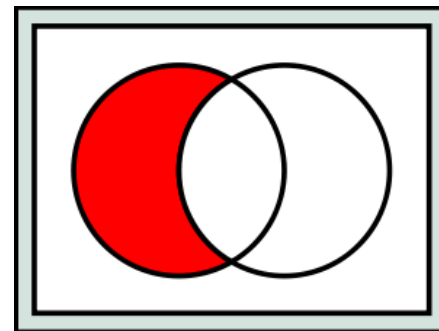
$\Pi_{\text{Cod_Reserva, Fecha_Hist}} (\sigma_{\text{Fecha_Hist} = '01/04/2005'} (\text{Reserva_Historica}))$

Álgebra Relacional – Diferencia

Diferencia (-)

Entrega todas aquellas tuplas que estan en R, pero no en S. R y S deben ser relaciones compatibles:

R - S



Por Ej:

Reserva - Reserva_Historica

Álgebra Relacional -> SQL

Diferencia (-)

```
SELECT Cod_Reserva, Fecha  
FROM Reserva
```

EXCEPT

```
SELECT Cod_Reserva, Fecha_Hist  
FROM Reserva_Historica  
WHERE cod_escuela=100
```

Relaciones compatibles:

- Cantidad de Atributos
- Dominios Compatibles

$\Pi_{\text{Cod_Reserva, Fecha}}(\text{Reserva})$

-

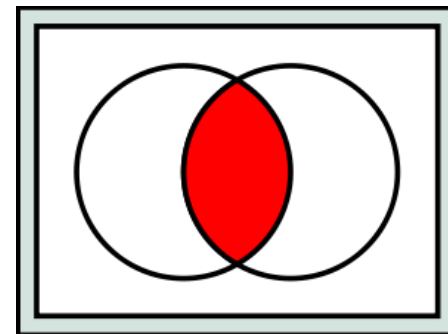
$\Pi_{\text{Cod_Reserva, Fecha_Hist}}(\sigma_{\text{Cod_escuela}=100}(\text{Reserva_Historica}))$

Álgebra Relacional – Intersección

Intersección ($\cap$)

La intersección, como en Teoría de conjuntos, corresponde al conjunto de todas las tuplas que están en R y en S, siendo R y S relaciones compatibles:

$$\mathbf{R \cap S}$$



Por Ej:

$$\mathbf{Reserva \cap Reserva_Historica}$$

Álgebra Relacional -> SQL

Intersección ($\cap$)

```
SELECT Cod_Reserva, Fecha  
FROM Reserva
```

INTERSECT

```
SELECT Cod_Reserva, Fecha_Hist  
FROM Reserva_Historica
```

Relaciones compatibles:

- **Cantidad de Atributos**
- **Dominios Compatibles**

$\Pi_{\text{Cod_Reserva, Fecha}}(\text{Reserva})$

$\cap$

$\Pi_{\text{Cod_Reserva, Fecha_Hist}}(\text{Reserva_Historica})$



Tipos de Reunión

Relación Instructor

ID_Instructor	Nombre	Nombre_departamento
112	Juan	Economía
124	José	Administración
136	María	Sistemas

Relación Enseña

ID_Instructor	Código de Curso
112	275/4
112	250/2
124	286/1
130	220/3

Álgebra Relacional – Producto Cartesiano

Producto cartesiano (x)

Entrega una relación, cuyo esquema corresponde a una combinación de todas las tuplas de R con cada una de las tuplas de S, y sus atributos corresponden a los de R seguidos por los de S:

R x S

Por Ej:

Instructor x Enseña



Tipos de Reunión

Producto Cartesiano

Instructor × Enseña

ID_Instructor	Nombre	Nombre_departamento	ID_Instructor	Código de Curso
112	Juan	Economía	112	275/4
124	José	Administración	112	275/4
136	María	Sistemas	112	275/4
112	Juan	Economía	112	250/2
124	José	Administración	112	250/2
136	María	Sistemas	112	250/2
112	Juan	Economía	124	286/1
124	José	Administración	124	286/1
136	María	Sistemas	124	286/1
112	Juan	Economía	130	220/3
124	José	Administración	130	220/3
136	María	Sistemas	130	220/3

Álgebra Relacional -> SQL

Producto cartesiano (x)

```
SELECT I.ID_Instructor, I.Nombre, I.Nombre_Departamento,  
       C.Id_Instructor, C.Cod_Curso  
FROM Instructor I, Enseña E
```

$\Pi_{\text{id_instructor, nombre...}}(\mathbf{p_i(Instructor)} \times \mathbf{p_e(Enseña)})$

Producto cartesiano (condicionado)

```
SELECT I.ID_Instructor, I.Nombre, I.Nombre_Departamento,  
       C.Id_Instructor, C.Cod_Curso  
FROM Instructor I, Enseña E  
WHERE I.Id_Instructor = E.Id_Instructor
```

$\Pi_{\text{id_instructor, nombre...}}(\sigma_{\mathbf{i.Id_Instructor = e.Id_Instructor}}(\mathbf{p_i(Instructor)} \times \mathbf{p_e(Enseña)}))$

Álgebra Relacional – Reunión Natural

Reunión Natural ($\bowtie$):

Hace un producto cartesiano de sus dos argumentos y realiza una selección forzando la igualdad de atributos que aparecen en ambas relaciones, eliminando repetidos:

$$\mathbf{R} \bowtie \mathbf{S}$$

Por Ej:

$$\begin{array}{l} \mathbf{p_i(Instructor)} \\ \mathbf{p_e(Enseña)} \end{array} \bowtie (i.Id_Instructor = e.Id_Instructor)$$



Tipos de Reunión

Reunión Natural

Instructor $\bowtie$ Enseña

ID_Instructor	Nombre	Nombre_departamento	ID_Instructor	Código de Curso
112	Juan	Economía	112	275/4
112	Juan	Economía	112	250/2
124	José	Administración	124	286/1

Álgebra Relacional -> SQL

Reunión Natural ($\bowtie$):

```
SELECT I.ID_Instructor, I.Nombre, I.Nombre_Departamento,  
       C.Id_Instructor, C.Cod_Curso  
FROM Instructor I INNER JOIN Enseña E  
      ON I.Id_Instructor = E.Id_Instructor
```

$\Pi_{\text{id_instructor, nombre...}} (p_i(\text{Instructor}) \bowtie_{(I.Id_Instructor = e.Id_Instructor)} p_e(\text{Enseña}))$

Álgebra Relacional – Reunión Externa

- Es una variante de la reunión (join) en la que se intenta mantener toda la información de los operandos, incluso para aquellas filas que no participan en la reunión.
- Se “rellenan con nulos” las tuplas que no tienen correspondencia en la reunión
- Tres variantes
 - Izquierda: se tienen en cuenta todas las filas del primer operando
 - Derecha: se tienen en cuenta todas las filas del segundo operando
 - Completa: se tienen en cuenta todas las filas de ambos operando

Tipos de Reunión

Reunión Externa Izquierda
Instructor $\bowtie$ Enseña

ID_Instructor	Nombre	Nombre_departamento	ID_Instructor	Código de Curso
112	Juan	Economía	112	275/4
112	Juan	Economía	112	250/2
124	José	Administración	124	286/1
136	María	Sistemas	NULL	NULL

Reunión Externa Derecha
Instructor $\ltimes$ Enseña

ID_Instructor	Nombre	Nombre_departamento	ID_Instructor	Código de Curso
112	Juan	Economía	112	275/4
112	Juan	Economía	112	250/2
124	José	Administración	124	286/1
NULL	NULL	NULL	130	220/3

Reunión Externa Completa
Instructor $\ltimes$ Enseña

ID_Instructor	Nombre	Nombre_departamento	ID_Instructor	Código de Curso
112	Juan	Economía	112	275/4
112	Juan	Economía	112	250/2
124	José	Administración	124	286/1
136	María	Sistemas	NULL	NULL
NULL	NULL	NULL	130	220/3

Tipos de Reunión

Reunión Externa:

```
SELECT I.ID_Instructor, I.Nombre, I.Nombre_Departamento,  
C.Id_Instructor, C.Cod_Curso AS "Codigo de Curso"
```

```
FROM Instructor I
```

```
LEFT JOIN Curso C ON C.IdInstructor = I.IdInstructor
```

```
RIGHT JOIN Curso C ON C.IdInstructor = I.IdInstructor
```

```
FULL JOIN Curso C ON C.IdInstructor = I.IdInstructor
```

Álgebra Relacional -> SQL

```
SELECT tel.Telefono
FROM Escuela esc, Telefono_Escuela tel
WHERE esc.codigo_escuela = tel.codigo_escuela
```

Producto cartesiano (x)

$\Pi_{\text{telefono}} (\sigma_{\text{esc.codigo_escuela} = \text{tel.codigo_escuela}} (\rho_{\text{esc}}(\text{Escuela}) \times \rho_{\text{tel}}(\text{Telefono_Escuela})))$

Reunión Natural ($\bowtie$)

1. Producto cartesiano (x)
2. Selección (σ) forzando la igualdad de atributos que aparecen en ambas relaciones, eliminando repetidos

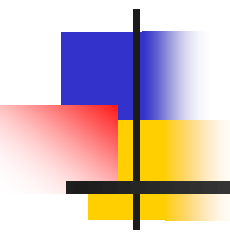
EQUIVALENTES

```
SELECT tel.Telefono
FROM Escuela esc INNER JOIN Telefono_Escuela tel ON
esc.codigo_escuela = tel.codigo_escuela
```

Reunión Natural ($\bowtie$)

$\Pi_{\text{telefono}} (\rho_{\text{esc}}(\text{Escuela}) \bowtie_{(\text{esc.codigo_escuela} = \text{tel.codigo_escuela})} \rho_{\text{tel}}(\text{Telefono_Escuela}))$

Manipulación, agregación y otras operaciones



Manipulación de Datos

Consultas de modificación de datos:

- **INSERT INTO** (neto positivo)
 - **VALUES**
 - **SELECT**
- **DELETE FROM** (neto negativo)
- **UPDATE** (neto cero)
 - Simple
 - Multiple
- Compatibles con Joins y Subconsultas
- Irreversibles (exceptuando **TRANSACTIONS**)
- Unarias – Solo afectan una tabla a la vez

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	6.00
5	5	Las cenizas del meteorito mortal	8.00
6	6	Visita a los huevos de mama Dino	10.00
7	7	Dinos argentinos	6.00
8	8	Zoo	11.00
9	9	Plantas exoticas	12.00
10	10	Mini explotadores, buscando dinos	9.00

INSERT INTO – Inserción

INSERT INTO *tabla* (col1, col2,... colN)
VALUES (valor1, valor2,... valorN)

- Definición de columnas opcional
 - Puede elegirse el orden o cantidad, obviando aquellas sin constraints NOT NULL
 - No especificar columnas causa que el DBMS espere todas las columnas en orden
- Definición de valores según tipo de dato
 - Cadenas de texto y fechas -> 'Amarillo' / '20241231'
 - Números enteros -> 54 / 1000001
 - Ceros por delante = texto -> 54 <> '00054'
 - Datos son "compatibilizados" de ser necesario
- Check de PRIMARY y FOREIGN KEYS al insertar

INSERT INTO – Inserción

INSERT INTO *Tipo_Visita* (Codigo, Descripcion, Arancel)
VALUES (11, 'Jardin Botanico', 18.00),
(12, 'El Mundo de Avatar', 20.00)

- Insertar mas de un registro
 - Una consulta, VALUES separados por comas
 - Múltiples consultas individuales
- Insertar resultado de una consulta
 - Reemplazando VALUES con una consulta SELECT
 - La forma de la consulta debe coincidir con el destino

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurus	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	6.00
5	5	Las cenizas del meteorito mortal	8.00
6	6	Visita a los huevos de mama Dino	10.00
7	7	Dinos argentinos	6.00
8	8	Zoo	11.00
9	9	Plantas exoticas	12.00
10	10	Mini explosores, buscando dinos	9.00
11	11	Jardin Botanico	18.00
12	12	El mundo de Avatar	20.00

DELETE - Eliminación

DELETE FROM *tabla*
WHERE *condición*

- Condición WHERE opcional
 - Mismo criterio que consultas SELECT
 - No usar condiciones actúa similar a un TRUNCATE TABLE
- Debe respetar integridad referencial
 - De existir constraints FK, la eliminación resulta en un error
 - Alternativas:
 - Eliminar registros en tablas "hijo" primero
 - Eliminar constraints FK
 - Aplicar configuración ON DELETE CASCADE

DELETE – Eliminación

DELETE FROM *Tipo_Visita*
WHERE Codigo **IN** (5, 7, 9)

- Recomendable usar PK en WHERE
 - Usar operadores como LIKE o campos con valores potencialmente repetidos hace posible eliminar registros equivocados
 - Usar PRIMARY KEY como condición asegura una eliminación única y precisa

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	6.00
5	6	Visita a los huevos de mama Dino	10.00
6	8	Zoo	11.00
7	10	Mini explotadores, buscando dinos	9.00
8	11	Jardin Botanico	18.00
9	12	El mundo de Avatar	20.00

UPDATE - Actualización

UPDATE *tabla1*
SET *campo = valor*

- Condición WHERE opcional
 - Mismo criterio que consultas SELECT
 - No usar condiciones realiza la actualización a lo largo de TODA la tabla
- Nuevos valores pueden tener diferentes orígenes
 - Valor fijo (5, 'Juan Perez', etc.)
 - Valor calculable (Arancel * 1.5)
 - Mediante JOINS
 - Clausula FROM seguida a SET
 - La tabla que se actualizara es la de la clausula UPDATE

UPDATE - Actualización

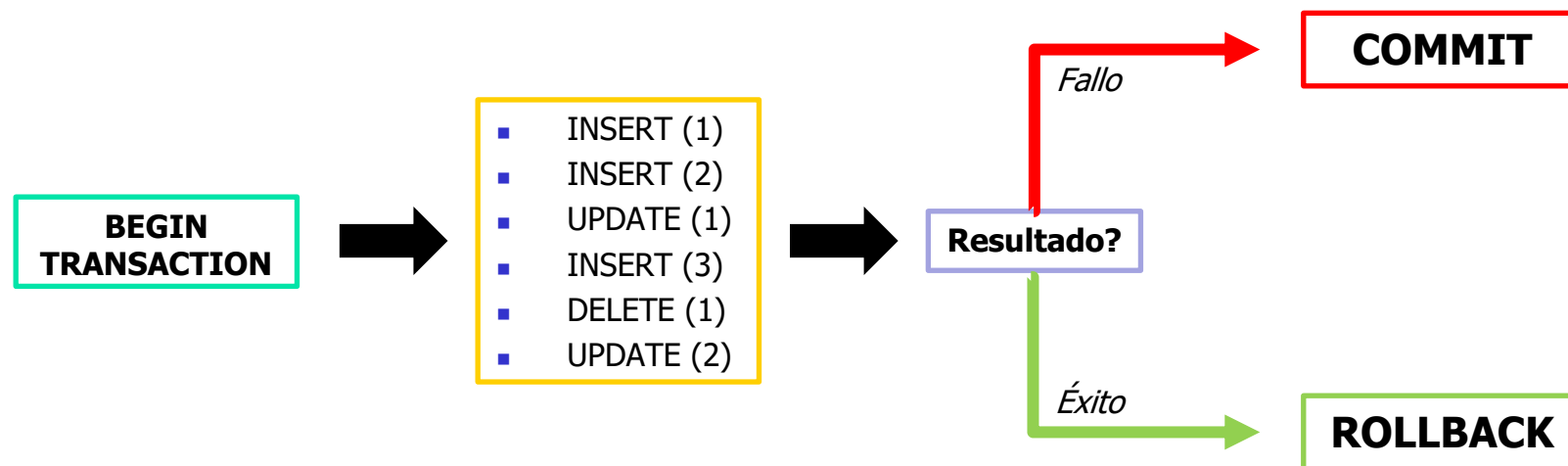
UPDATE *Tipo_Visita*
SET *Arancel = 7.50*
WHERE *Codigo < 4*

- Actualizar mas de un campo
 - Una consulta, igualaciones separadas por coma (Ej: Descripcion = 'Zoo 2', Arancel = 15.00)
 - Múltiples consultas individuales

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	7.50
2	2	Los Mamuts en Familia	7.50
3	3	Las Tortugas Prehistoricas	7.50
4	4	Laguna de Nahuelito	6.00
5	6	Visita a los huevos de mama Dino	10.00
6	8	Zoo	11.00
7	10	Mini exploradores, buscando dinos	9.00
8	11	Jardin Botanico	18.00
9	12	El mundo de Avatar	20.00

Transacciones

Ante la ejecución de una o múltiples operaciones de modificación de datos sensibles (incluyendo modificar objetos de la base mediante DDL), pueden utilizarse transacciones para establecer puntos de “restauración”



Mientras una transacción se encuentre activa, múltiples objetos de la base son aislados localmente para quien la inicio, bloqueándolos a otros usuarios y procesos.

Transacciones

Iniciamos una transacción, pero afectamos mas registros de los que queremos:

BEGIN TRAN

UPDATE *Tipo_Visita*

SET *Arancel = 2*

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	2.00
2	2	Los Mamuts en Familia	2.00
3	3	Las Tortugas Prehistoricas	2.00
4	4	Laguna de Nahuelito	2.00
5	5	Las cenizas del meteorito mortal	2.00
6	6	Visita a los huevos de mama Dino	2.00
7	7	Dinos argentinos	2.00

Mediante el comando ROLLBACK, podemos revertir todas las operaciones realizadas hasta el principio de la transacción, lo cual la finaliza:

ROLLBACK

SELECT *
FROM *Tipo_visita*

Results Messages			
(7 rows affected)			
	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	6.00
5	5	Las cenizas del meteorito mortal	8.00
6	6	Visita a los huevos de mama Dino	10.00
7	7	Dinos argentinos	6.00

Transacciones

Después de arreglar nuestra consulta, iniciamos una transacción nuevamente:

BEGIN TRAN

UPDATE *Tipo_Visita*

SET *Arancel = 2*

WHERE *Codigo IN (4, 5, 7)*

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	2.00
5	5	Las cenizas del meteorito mortal	2.00
6	6	Visita a los huevos de mama Dino	10.00
7	7	Dinos argentinos	2.00

Mediante el comando COMMIT, confirmamos todas las operaciones realizadas desde el principio de la transacción y la finalizamos:



(3 rows affected)

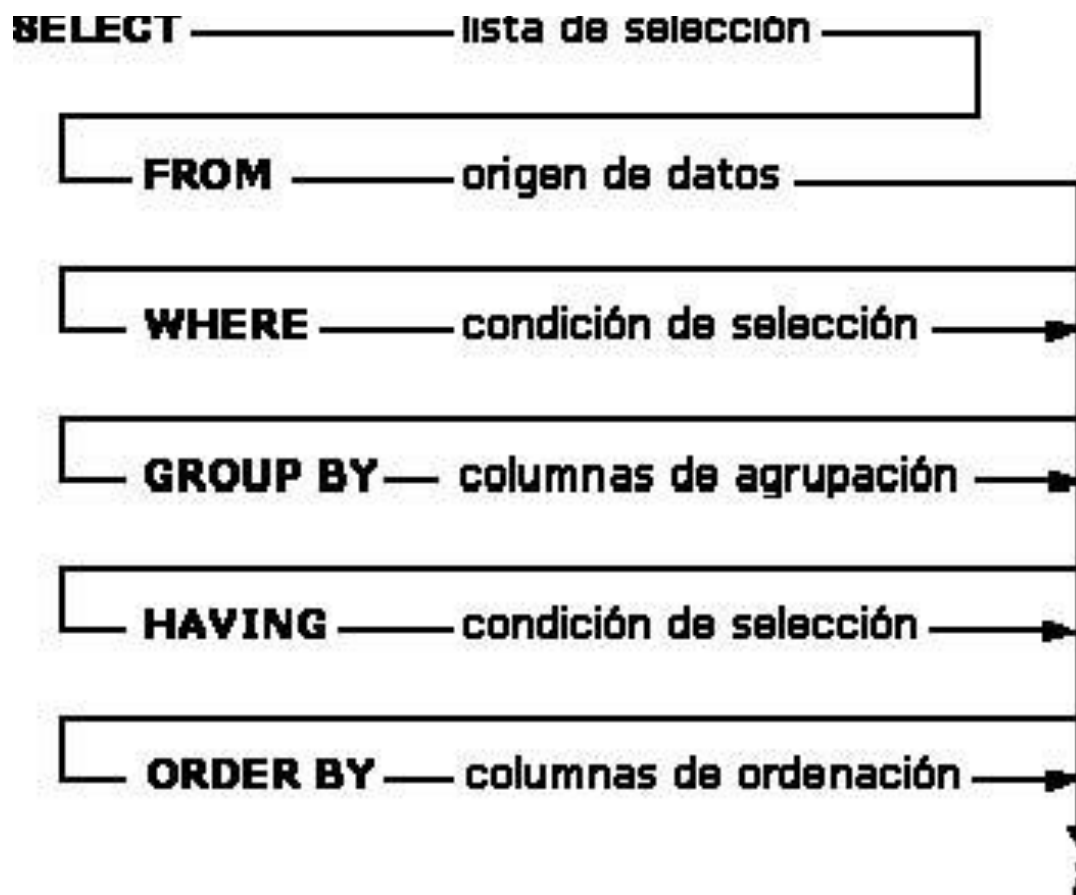
COMMIT

SELECT *

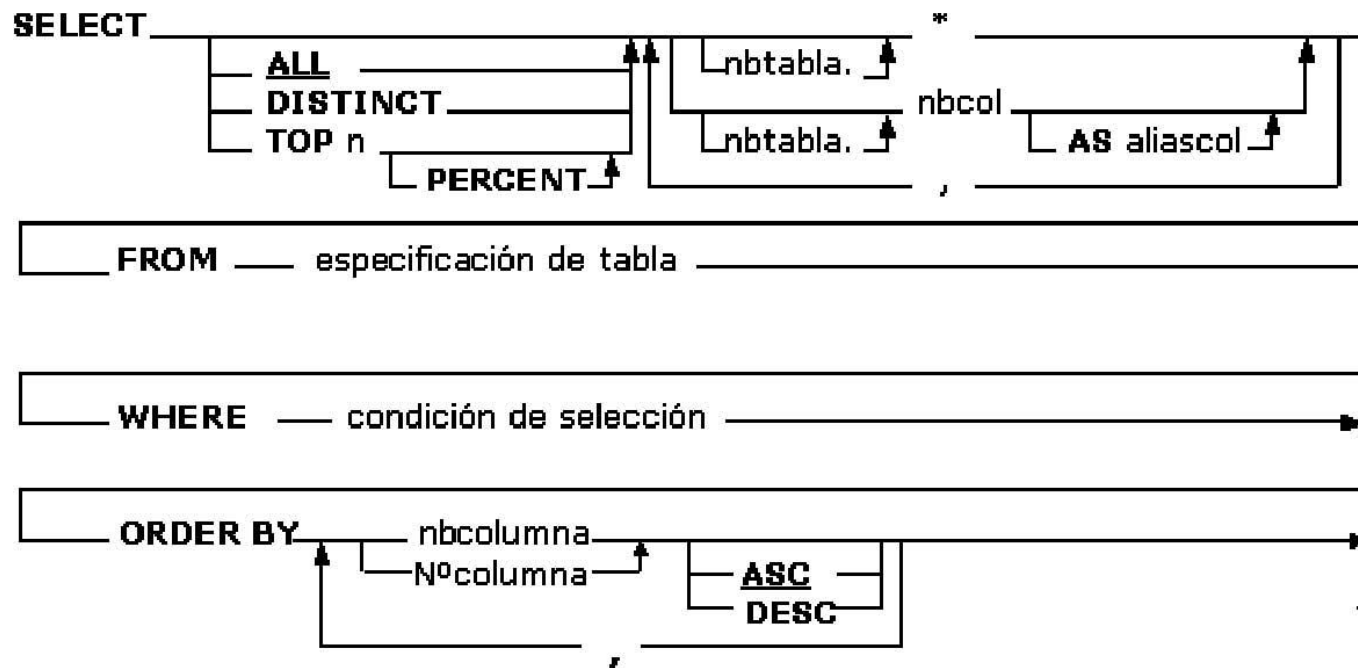
FROM *Tipo_visita*

	Codigo	Descripcion	Arancel
1	1	Parque de Brontosaurios	10.00
2	2	Los Mamuts en Familia	8.00
3	3	Las Tortugas Prehistoricas	4.00
4	4	Laguna de Nahuelito	2.00
5	5	Las cenizas del meteorito mortal	2.00
6	6	Visita a los huevos de mama Dino	10.00
7	7	Dinos argentinos	2.00

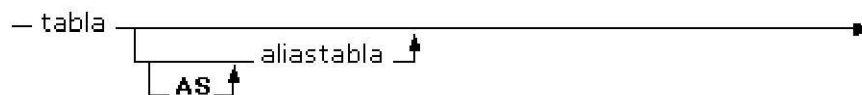
SQL Básico: Consultas de Resumen



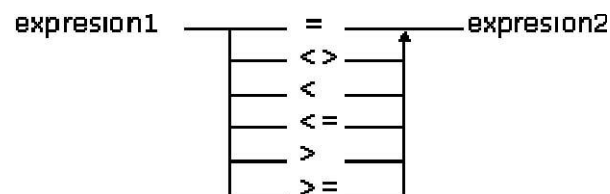
SQL Básico: SELECT



Especificación de tabla



Condición de selección



Distinto **<>** (estandar)
=! (no estandar)
 ¡No aplica para NULL!
"IS NOT NULL"
"IS NULL"

SELECT - Partes y Orden de resolución

SQL

Álgebra Relacional

- SELECT **Nombre_escuela, COUNT (Codigo_escuela)**
Proyección
- FROM **Escuela**
Definición de Relaciones (renombramiento, reunión natural, etc.)
- WHERE **Nombre_escuela Like 'N%'**
Selección
- GROUP BY **Nombre_escuela**
Reagrupamiento
- HAVING **COUNT (*) > 3**
Filtrado de grupos
- ORDER BY **Nombre_escuela**
Ordenamiento

Aritmética

SQL puede realizar operaciones matemáticas en diferentes clausulas dentro de una consulta. Las operaciones requieren un tipo de dato numérico.

- Operadores básicos:

- Suma (+)
- Diferencia (-)
- Multiplicacion (*)
- Division (/)

- Pueden realizarse sobre datos en tablas, o usando valores definidos por el usuario

SELECT Escuela,
CantAlumnos,
CantAlumnos + 50

FROM DetalleVisitas

	Escuela	CantAlumnos	CantAlumnosExtra
1	Escuela Domingo Faustino Sarmiento	25	75
2	Escuela Domingo Faustino Sarmiento	21	71
3	Escuela Domingo Faustino Sarmiento	89	139
4	Instituto Santa Elena	50	100
5	Instituto Santa Elena	53	103
6	Instituto Santa Elena	78	128

Aritmética

- Operaciones entre campos de una tabla:

SELECT

Escuela,
Reservas,
CantAlumnos,
(Reservas – CantAlumnos),
(CantAlumnos / Reservas) * 100

FROM

DetalleVisitas

	Escuela	Reservas	CantAlumnos	Diferencia	Asistencia
1	Escuela Domingo Faustino Sarmiento	30	25	5	83.33333300
2	Escuela Domingo Faustino Sarmiento	25	21	4	84.00000000
3	Escuela Domingo Faustino Sarmiento	110	89	21	80.90909000
4	Instituto Santa Elena	56	50	6	89.28571400
5	Instituto Santa Elena	100	53	47	53.00000000
6	Instituto Santa Elena	90	78	12	86.66666600

Aritmética

- Operaciones entre campos de diferentes tablas:

SELECT

Escuela,
DV.Visita,
CantAlumnos,
ArancelPorAlumno,
(CantAlumnos * ArancelPorAlumno)

FROM

DetalleVisitas DV **INNER JOIN** Aranceles A

ON DV. Visita = A.Visita

	Escuela	Visita	CantAlumnos	ArancelPorAlumno	ArancelTotal
1	Escuela Nacional Nro 4 Nicolas Avellaneda	1	103	10.00	1030.00
2	Escuela Nacional Nro 4 Nicolas Avellaneda	2	125	8.00	1000.00
3	Escuela Nacional Nro 4 Nicolas Avellaneda	1	74	10.00	740.00
4	Escuela Nacional Nro 4 Nicolas Avellaneda	7	38	6.00	228.00
5	First Escuela Super Important	1	20	10.00	200.00
6	First Escuela Super Important	2	26	8.00	208.00
7	First Escuela Super Important	11	30	18.00	540.00

GROUP BY - Agregación

SQL permite hacer transformaciones internas de tablas, realizando agrupaciones de registros para conseguir datos calculados a través de funciones de agregación

SELECT *Col1, (Función)*
FROM *Tabla*
GROUP BY *Col1*

■ Clausula GROUP BY

- Permite elegir columnas para conformar grupos de datos calculados
- Mas columnas en GROUP BY = Mas segmentación de grupos

	Escuela	NumReserva	Visita	CantAlumnos
1	Escuela Nacional Nro 4 Nicolas Avellaneda	4	1	120
2	Escuela Nacional Nro 4 Nicolas Avellaneda	4	2	130
3	Escuela Nacional Nro 4 Nicolas Avellaneda	12	1	91
4	Escuela Nacional Nro 4 Nicolas Avellaneda	12	7	45
5	First Escuela Super Important	8	1	25
6	First Escuela Super Important	8	2	32
7	First Escuela Super Important	61	11	88
8	Instituto Santa Elena	2	1	56
9	Instituto Santa Elena	2	3	100
10	Instituto Santa Elena	2	4	90
11	Instituto Santa Elena	6	1	93
12	Instituto Santa Elena	6	5	65
13	Instituto Santa Elena	6	14	37
14	Instituto Santa Elena	11	5	87
15	Instituto Santa Elena	11	6	30
16	Instituto Santa Elena	63	5	58
17	Instituto Santa Elena	63	15	85

GROUP BY - Agregación

Al usar GROUP BY, toda la información de la tabla que no corresponda a columnas en esa clausula, se “aplasta” a fines de realizar agregaciones.

SELECT

Escuela, NumReserva

FROM

DetallesVisitas

GROUP BY

Escuela, NumReserva

	Escuela	NumReserva	Visita	CantAlumnos
1	Escuela Nacional Nro 4 Nicolas Avellaneda	4	1	120
2	Escuela Nacional Nro 4 Nicolas Avellaneda	4	2	130
3	Escuela Nacional Nro 4 Nicolas Avellaneda	12	1	91
4	Escuela Nacional Nro 4 Nicolas Avellaneda	12	7	45
5	First Escuela Super Important	8	1	25
6	First Escuela Super Important	8	2	32
7	First Escuela Super Important	61	11	88
8	Instituto Santa Elena	2	1	56
9	Instituto Santa Elena	2	3	100
10	Instituto Santa Elena	2	4	90
11	Instituto Santa Elena	6	1	93
12	Instituto Santa Elena	6	5	65
13	Instituto Santa Elena	6	14	37
14	Instituto Santa Elena	11	5	87
15	Instituto Santa Elena	11	6	30
16	Instituto Santa Elena	63	5	58
17	Instituto Santa Elena	63	15	85



	Escuela	NumReserva
1	Escuela Nacional Nro 4 Nicolas Avellaneda	4
2	Escuela Nacional Nro 4 Nicolas Avellaneda	12
3	First Escuela Super Important	61
4	First Escuela Super Important	8
5	Instituto Santa Elena	11
6	Instituto Santa Elena	6
7	Instituto Santa Elena	63
8	Instituto Santa Elena	2

GROUP BY - Agregación

Una columna que no se encuentre en la cláusula **GROUP BY**, no puede incluirse en la cláusula **SELECT**



SELECT
FROM
GROUP BY

Escuela, NumReserva, Visita
DetallesVisitas
Escuela, NumReserva



SELECT
FROM
GROUP BY

Escuela, NumReserva, CantAlumnos
DetallesVisitas
Escuela



SELECT
FROM
GROUP BY

Escuela
DetallesVisitas
Escuela, NumReserva

GROUP BY - Agregación

Funciones de agregación:

Función	Uso
COUNT (* / Col)	Cuenta cantidad de registros dentro de un grupo Puede usar DISTINCT para contar ocurrencias únicas
SUM (Col)	Suma todos los valores de la columna del grupo
AVG (Col)	Promedia todos los valores de la columna del grupo
MAX (Col)	Obtiene el mínimo valor de la columna del grupo
MIN (Col)	Obtiene el máximo valor de la columna del grupo

Las funciones pueden emplearse a nivel Tabla o a nivel Grupo:

Tabla		Grupo	
SELECT	COUNT(*)	SELECT	Sexo, COUNT(*)
FROM	Empleados	FROM	Empleados
		GROUP BY	Sexo

GROUP BY - Agregación

SELECT Escuela, **COUNT**(*)
FROM DetalleVisitas
GROUP BY Escuela

	Escuela	CantidadVisitas
1	Escuela Nacional Nro 4 Nicolas Avellaneda	4
2	First Escuela Super Important	3
3	Instituto Santa Elena	10

SELECT Escuela, NumReserva,
COUNT(*)
FROM DetalleVisitas
GROUP BY Escuela, NumReserva

	Escuela	NumReserva	CantidadVisitas
1	Escuela Nacional Nro 4 Nicolas Avellaneda	4	2
2	Escuela Nacional Nro 4 Nicolas Avellaneda	12	2
3	First Escuela Super Important	8	2
4	First Escuela Super Important	61	1
5	Instituto Santa Elena	2	3
6	Instituto Santa Elena	6	3
7	Instituto Santa Elena	11	2
8	Instituto Santa Elena	63	2

SELECT Escuela, **COUNT**
(DISTINCT TipoVisita)
FROM DetalleVisitas
GROUP BY Escuela

	Escuela	CantidadVisitas
1	Escuela Nacional Nro 4 Nicolas Avellaneda	3
2	First Escuela Super Important	3
3	Instituto Santa Elena	7

GROUP BY - Agregación

- Múltiples funciones

- Una consulta puede incluir varias funciones, las cuales calculan datos en base al mismo grupo

SELECT Escuela, **SUM**(CantAlumnos), **AVG**(CantAlumnos),
MIN(CantAlumnos), **MAX**(CantAlumnos)
FROM DetalleVisitas
GROUP BY Escuela

	Escuela	TotalAlumnos	PromedioAlumnos	MinAlumnos	MaxAlumnos
1	Escuela Nacional Nro 4 Nicolas Avellaneda	386	96	45	130
2	First Escuela Super Important	145	48	25	88
3	Instituto Santa Elena	701	70	30	100

GROUP BY - Agregación

■ Clausula HAVING

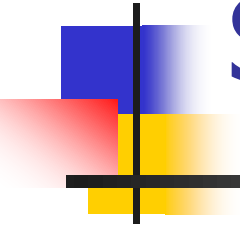
- SELECT -> FROM -> WHERE -> GROUP BY -> HAVING
- Actúa como un WHERE, post-agrupamiento. Útil para condiciones usando funciones.
- Opcional, no es obligatoria al usar GROUP BY

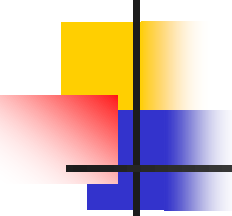
SELECT Escuela, **SUM**(CantAlumnos), **AVG**(CantAlumnos),
MIN(CantAlumnos), **MAX**(CantAlumnos)
FROM DetalleVisitas
GROUP BY Escuela
HAVING SUM(CantAlumnos) > 300

	Escuela	TotalAlumnos	PromedioAlumnos	MinAlumnos	MaxAlumnos
1	Escuela Nacional Nro 4 Nicolas Avellaneda	386	96	45	130
2	First Escuela Super Important	145	48	25	88
3	Instituto Santa Elena	701	70	30	100

Subconsultas

Stored Procedures - Triggers

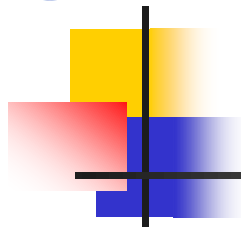




Subconsultas

Es una consulta anidada en una instrucción SELECT, INSERT, UPDATE o DELETE, o bien en otra subconsulta.

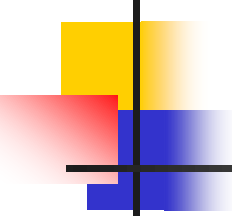
- Aparecen siempre entre paréntesis
- Sólo pueden incluir ORDER BY cuando incluyen una cláusula TOP



Subconsultas

Las subconsultas en la instrucción SELECT, pueden estar:

- En la lista de selección SELECT
- En el FROM
- Como condición en el WHERE o en el HAVING
 - Operadores Lógicos ($>$, $<$, $=$, $<>$)
 - IN (not IN)
 - ANY-ALL
 - EXISTS (not exists)



Subconsultas

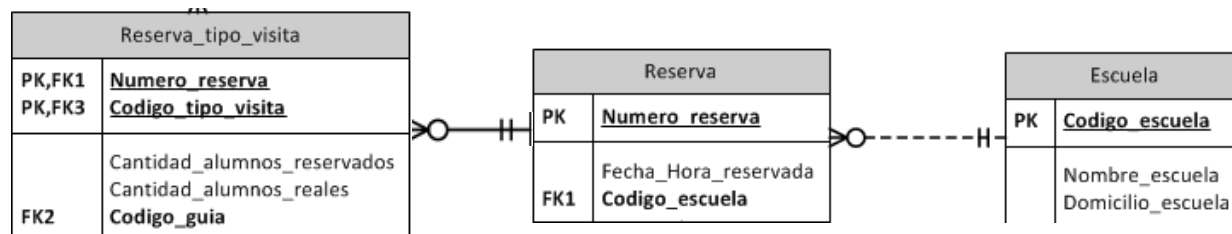
- Referencia Externa

Es un nombre de columna que no se refiere a ninguna de las tablas de la cláusula FROM de la subconsulta, sino a las de la consulta principal.

Como la subconsulta se ejecuta una vez por cada fila de la principal, el valor de la referencia externa va cambiando.

Ejemplos - FROM

Listar nombre de las escuelas y cantidad total de alumnos reservados para su próxima visita.



```
SELECT E.Nombre_escuela,  
        SUM(RTV.Cantidad_alumnos_reservados) as Total  
FROM   Escuela E INNER JOIN Reserva R  
        ON R.Codigo_escuela = E.Codigo_escuela  
        INNER JOIN Reserva_tipo_visita RTV  
        ON RTV.Numero_reserva = R.Numero_reserva  
GROUP BY E.Nombre_escuela
```


Ejemplos - FROM

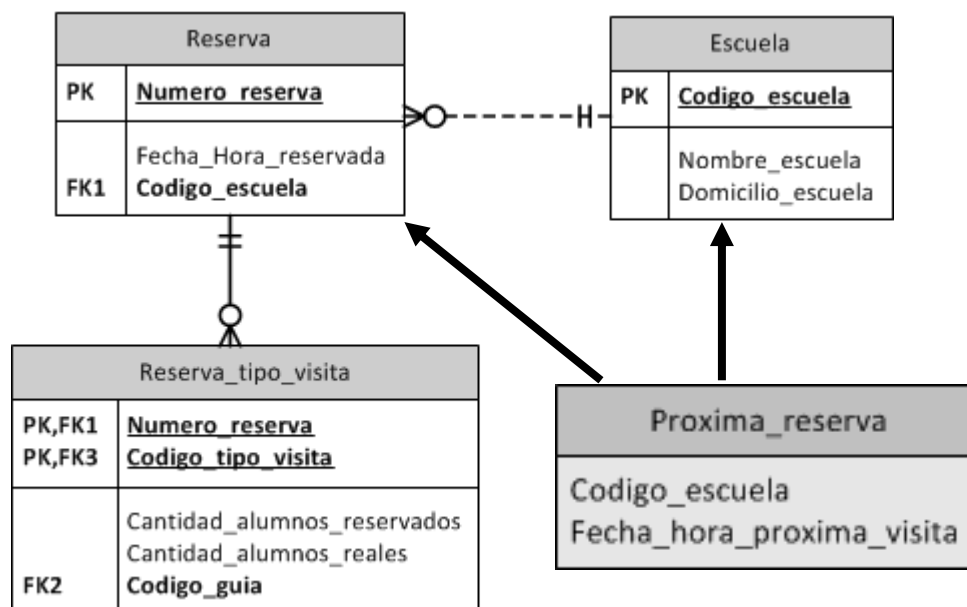
Listar nombre de las escuelas y cantidad total de alumnos reservados para su próxima visita.

Proxima_reserva
Codigo_escuela
Fecha_hora_proxima_visita

```
SELECT Codigo_escuela,  
        MIN(Fecha_Hora_reservada) AS 'Fecha_hora_proxima_visita'  
FROM   Reserva  
WHERE  Fecha_Hora_reservada > Getdate()  
GROUP BY Codigo_escuela
```

Ejemplos - FROM

Listar nombre de las escuelas y cantidad total de alumnos reservados para su próxima visita.



Ejemplos - FROM

Listar nombre de las escuelas y cantidad total de alumnos reservados para su próxima visita.

```
SELECT  E.Nombre_escuela,  
          SUM(RTV.Cantidad_alumnos_reservados)  
FROM    Escuela E INNER JOIN Reserva R  
          ON R.Codigo_escuela = E.Codigo_escuela  
          INNER JOIN Reserva_tipo_visita RTV  
          ON RTV.Numero_reserva = R.Numero_reserva  
          INNER JOIN  
          (SELECT  Codigo_escuela,  
                  Min(Fecha_Visita_reservada) AS 'Fecha_hora_proxima_visita'  
                  FROM    Reserva  
                  WHERE   Fecha_Visita_reservada > Getdate()  
                  GROUP BY código_escuela) AS Proxima_Reserva  
          ON Proxima_reserva.Codigo_escuela = E.Codigo_escuela  
          AND Proxima_reserva.Fecha_hora_proxima_visita = R.Fecha_hora_reservada  
GROUP BY E.Nombre_escuela
```

Ejemplos – WHERE (valor s/ref)

Listar los códigos de guías que hayan guiado, al menos, una visita cuya cantidad de alumnos reservados haya superado al promedio de cantidad de alumnos reservados para una visita.

```
SELECT DISTINCT RTV1.codigo_guia  
FROM Reserva_tipo_visita RTV1  
WHERE cantidad_alumnos_reservados >  
      (SELECT AVG(cantidad_alumnos_reservados)  
       FROM reserva_tipo_visita)
```

Operador de comparación >:
La subconsulta reemplaza
un valor, por lo que debe
devolver sólo un valor

Ejemplos – WHERE (valor c/ref)

Listar los códigos de guías que hayan guiado, al menos, una visita cuya cantidad de alumnos reservados haya superado al promedio de cantidad de alumnos reservados para las visitas de ese tipo.

```
SELECT DISTINCT RTV1.Codigo_Guia  
FROM Reserva_Tipo_Visita RTV1  
WHERE RTV1.Cantidad_Alumnos_Reservados >
```

```
(SELECT AVG(RTV2.Cantidad_Alumnos_Reservados)  
FROM Reserva_Tipo_Visita RTV2  
WHERE RTV2.Código_Tipo_Visita = RTV1.Código_Tipo_Visita  
GROUP BY RTV2.Código_Tipo_Visita)
```

Operador de comparación >:
La subconsulta reemplaza
un valor, por lo que debe
devolver sólo un valor

Referencia Externa

Ejemplos – WHERE (subconjunto)

Listar guías que hayan guiado a visitas con 40 alumnos o más.

```
SELECT Codigo_Guia,  
        Apellido_Guia,  
        Nombre_Guia  
FROM   Guia  
WHERE  Codigo_Guia IN  
        (SELECT rtv.codigo_guia  
         FROM Reserva_Tipo_Visita rtv  
         WHERE rtv.Cantidad_Alumnos_Reales >= 40)
```

Puede tener referencia externa.
En ese caso, el subconjunto se calcula
para cada fila.

Operador IN: aplica a listas
De valores

Ejemplos - SELECT

Listar todos los tipos de visita en conjunto con dos columnas que indiquen, respectivamente, la cantidad de veces que los guías con código 1 y 2 dieron esos paseos.

Restricción:
La subconsulta debe devolver
Una fila y una columna

Reserva_tipo_visita	
PK,FK1	<u>Numero_reserva</u>
PK,FK3	<u>Codigo_tipo_visita</u>
FK2	Cantidad_alumnos_reservados Cantidad_alumnos_reales <u>Codigo_guia</u>



tipo_visita	
PK,FK1	<u>Codigo_tipo_visita</u>
FK1	Descripcion_tipo_visita Arancel_por_alumno Numero_reserva

```
SELECT    Codigo_Guia, COUNT(*) AS Cantidad
FROM      Reserva_tipo_visita RTV
WHERE      RTV.Codigo_Guia = 1
AND        RTV.cantidad_alumnos_reales IS NOT NULL
GROUP BY  RTV.Codigo_Guia
```

```
SELECT    Codigo_Guia, COUNT(*) AS Cantidad
FROM      Reserva_tipo_visita RTV
WHERE      RTV.Codigo_Guia = 2
AND        RTV.cantidad_alumnos_reales IS NOT NULL
GROUP BY  RTV.Codigo_Guia
```

Ejemplos - SELECT

Listar todos los tipos de visita en conjunto con dos columnas que indiquen, respectivamente, la cantidad de veces que los guías con código 1 y 2 dieron esos paseos.

Restricción:
La subconsulta debe devolver
Una fila y una columna

```
SELECT TV.Descripcion_Tipo_Visita,
```

```
(SELECT COUNT(*)  
FROM Reserva_tipo_visita RTV1  
WHERE RTV1.Codigo_guia = 1  
AND RTV1.codigo_tipo_visita = TV.Codigo_tipo_visita  
AND RTV1.cantidad_alumnos_reales IS NOT NULL  
GROUP BY RTV1.Codigo_Guia) as 'Guia 1',
```

```
(SELECT COUNT(*)  
FROM Reserva_tipo_visita RTV2  
WHERE RTV2.Codigo_guia = 2  
AND RTV2.codigo_tipo_visita = TV.Codigo_tipo_visita  
AND RTV2.cantidad_alumnos_reales IS NOT NULL  
GROUP BY RTV2.Codigo_Guia) as 'Guia 2'
```

```
FROM Tipo_Visita TV
```


Ejemplos – INSERT

Inserto el resultado de una consulta.
Técnicamente no es una subconsulta
por eso no va entre ()

Una escuela quiere realizar una reserva para el 14/08/2016 con las mismas condiciones que tuvo en su reserva nº 50 (La última reserva registrada en el sistema tiene el número 124)

```
INSERT INTO Reserva
SELECT      125, '20160814', hora_visita_reservada, código_escuela
FROM        Reserva
WHERE       numero_reserva = 50
```

```
INSERT INTO Reserva_Tipo_Visita
SELECT      125, código_tipo_visita, cantidad_alumnos_reservados, NULL, NULL
FROM        Reserva_Tipo_Visita
WHERE       numero_reserva = 50
```

```
INSERT INTO Reserva_Tipo_Visita_Grado
SELECT      125, código_tipo_visita, numero_grado
FROM        Reserva_Tipo_Visita_Grado
WHERE       numero_reserva = 50
```

Subconsultas - Resumen

Subconsultas en el SELECT:
Sólo pueden devolver una columna
y una fila

Subconsultas en el FROM:
Se trata como una tabla. Se ejecuta
una sola vez antes de la consulta
Principal.

Subconsultas como condición (en el WHERE y en el HAVING):

- condiciones de comparación (=, <, >, <=, >=, <>)
- condiciones de comparación cuantificada (ANY, ALL)
- pertenencia a un conjunto (IN)
- existencia (EXISTS)

Se pueden utilizar en cualquier consulta que utilice la cláusula en cuestión (ya sea si es de consulta, actualización, inserción o eliminación).

Ejemplos: INSERT con SELECT, UPDATE con FROM, o DELETE con WHERE

Stored Procedures

- Un Procedimiento Almacenado es un programa escrito en lenguaje del DBMS, almacenado como parte de la Base de Datos.
 - Aceptan parámetros de entrada y entregan valores en forma de parámetros de salida.
 - Contienen instrucciones de programación que realizan operaciones en la base de datos
 - Manejan errores, devolviendo un valor de estado que indica si la operación se ha realizado correctamente o ha habido un error.

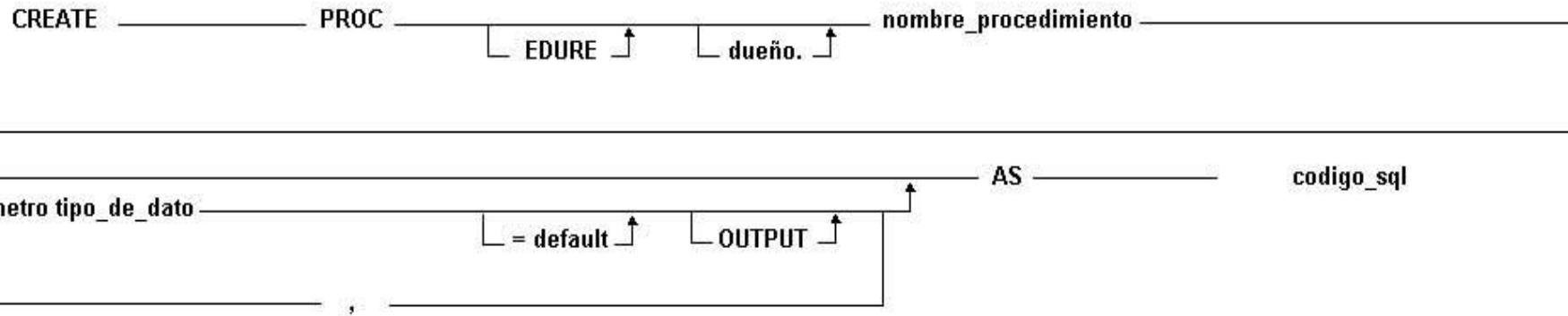
Stored Procedures

■ Ventajas

- Permiten una programación modular.
- Se puede crear el procedimiento una vez, almacenarlo en la base de datos y llamarlo desde el programa tantas veces como desee.
- Permiten una ejecución más rápida: Los procedimientos son analizados y optimizados después de haberlos ejecutado una primera vez, y es posible utilizar una versión compilada del procedimiento que se encuentra en la memoria.
- Pueden reducir el tráfico de red: Una operación que necesite centenares de líneas de código SQL puede realizarse mediante una sola instrucción que ejecute el código en un procedimiento, en vez de enviar cientos de líneas de código por la red.
- Pueden utilizarse como mecanismo de seguridad: Es posible conceder permisos a los usuarios para ejecutar un procedimiento almacenado.



Stored Procedures



Ejemplo - Stored Procedures

Crear un procedimiento almacenado que liste los nombres de las escuelas que comiencen con una cadena de texto determinada por un parámetro de entrada (con valor predeterminado: comienza con “Esc”).

```
CREATE PROCEDURE ListaEscuelas @Nombre char(20)
```

```
AS
```

```
SELECT Nombre_Escuela
```

```
FROM Escuela
```

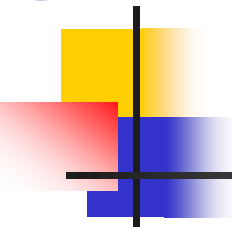
```
WHERE Nombre_Escuela LIKE @Nombre
```

Ejecucion: EXEC ListaEscuelas 'Esc%'



Triggers

- Un Triggers o disparador es una rutina autónoma asociada con una tabla o vista que automáticamente realiza una acción cuando se inserta (INSERT), se actualiza (UPDATE), o se borra (DELETE) una fila en la tabla.
- Un Trigger nunca se llama directamente.
 - Evento: es la modificación de datos que se controla con el trigger, y hará que este se dispare
 - Condición: debe verificarse para que el trigger se dispare.
 - Acción: La acción (normalmente una modificación de datos o evitar que la operación en curso se realice) que se produce como reacción de la BD ante el cambio.



Triggers

■ Ventajas

- Evitan el almacenamiento de información errónea en la base de datos.
- Aumentan la integridad referencial o conseguir una seguridad adicional.
- Controlar restricciones de usuario: por ejemplo, una regla de negocio de una empresa puede especificar que un empleado no puede ganar más que su jefe.
- Mantenimiento de valores deducidos: Si tenemos una tabla con valores parciales de ventas y otra con totales, al insertar, borrar o modificar un valor parcial, automáticamente se actualizará el valor total.
- Mantener coherentes las tablas replicadas en una BD distribuida.
- Registrar en tablas de auditoria las actualizaciones de la base de datos.

Triggers

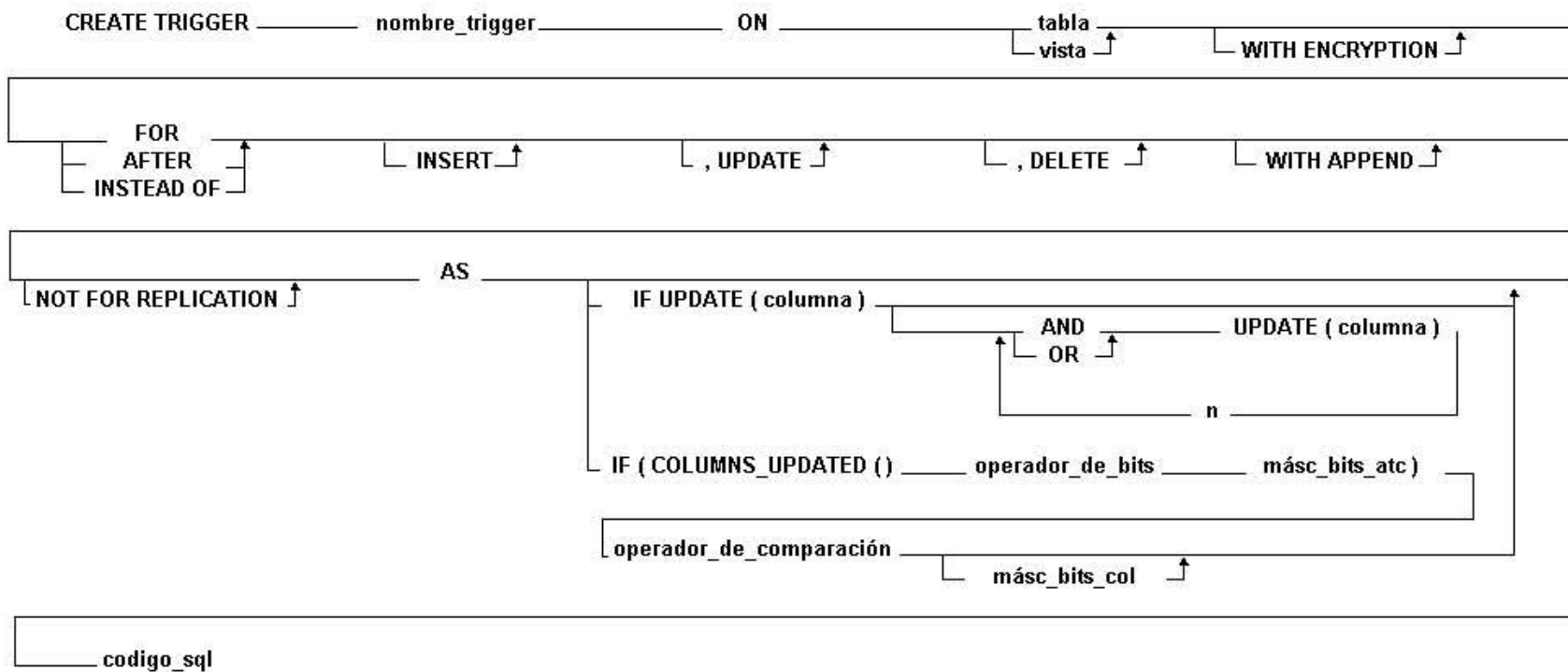


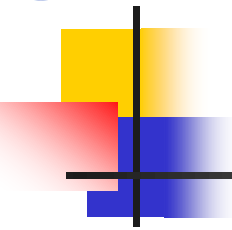
INSTEAD OF

AFTER

SQL Server: Tablas Inserted y Deleted

Triggers





Triggers

Crear un trigger sobre la tabla Reserva_Tipo_Visita que en caso de que se borre una fila de ésta elimine las respectivas filas de la tabla Guía.

```
CREATE TRIGGER trig_delete ON reserva_tipo_visita FOR DELETE
```

```
AS
```

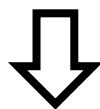
```
DELETE FROM Guia
```

```
WHERE codigo_guia IN
```

```
(SELECT codigo_guia FROM deleted ) ;
```

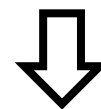
Resumen

Un Procedimiento Almacenado es un programa almacenado en la base para la realización de operaciones específicas.



- Parámetro IN-OUT
- Manejo de Errores
- Instrucciones de Programación

Un Trigger es una rutina autónoma asociada con una tabla o vista que realiza automáticamente una acción ante un determinado evento



- Reglas de negocio
- Validaciones
- Restricciones
- Integridad Referencial
- Auditoría